MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión B del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_sanchez_citlali
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/sanchez_citlali/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
En la radiografía de un objeto, ¿qué representa un atributo y qué representa un método? Da un ejemplo de cada uno.

Tu respuesta

```
un atributo esta dentro de la clase, es como las caracteristicas que tienes del objeto. Un método es una función, te explica que hacer y tiene parametros
```

Corrección. Bien: el atributo como característica del objeto y el método como lo que hace se acercan a la idea de estado frente a comportamiento. Faltó el ejemplo de cada uno que pedía la pregunta, por ejemplo el atributo velocidad de un coche y el método acelerar, que cambia esa velocidad.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

Tu respuesta

```
Es static porque no puede modificarse, el main es el principal y por lo tanto no debe modificarse y siempre se queda asi. Todos pueden acceder a ella.
```

Corrección. Decir que main es 'el principal' apunta a que es el punto de entrada del programa. Pero static no quiere decir que no se pueda modificar ni que todos accedan a él: quiere decir que el método pertenece a la clase y la JVM puede llamarlo sin crear antes una instancia.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un atributo de instancia y uno static? Explica qué comparten dos objetos de la misma clase.

Tu respuesta

```
Los atributos static estan definidos en la clase y todos pueden acceder a ellos, mientras que los de instancia estan en un metodo y cuando termina el metodo el atributo ya no se usa. Una clase es como un molde de dinosaurio y un objeto es el dinosaurio, si tienes dos dinosaurios ambos son diferentes objetos pero comparten el mismo molde y por lo tanto las mismas caracteristicas principales. Igual para eso se ocupan los constructores, para definir más objetos de la misma clase ouo
```

Corrección. Es válido que el atributo static se define en la clase y que todos lo usan. El error es que los atributos de instancia no viven dentro de un método (eso es una variable local): cada objeto tiene su propia copia, y lo que dos objetos comparten es el dato static de la clase, como un contador de objetos creados.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

Tu respuesta

```
una variable primitiva son números, por ejemplo: int, double, float.    Mientras que una variable de referencia te da una dirección, por ejemplo, String
```

Corrección. Bien: la variable de referencia guarda una dirección (String) y la primitiva guarda un valor (int, double), con ejemplos correctos. Le falta precisión: la primitiva guarda el valor directamente y no siempre es un número (boolean, char), y la referencia apunta a un objeto.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?

Tu respuesta

```
javac sirve para traducirle a la computadora lo del idioma humano a un idioma que ella habla, el jvm lo ejecuta, porque es java verification machine. Como la compu no entiende tiene que tener un traductor que lo traduza y luego eso lo vuelve a codificar a nuestro idioma.
```

Corrección. Es correcto que javac traduce el código y que la JVM lo ejecuta. Faltó decir que javac produce bytecode (.class). Hay un error de concepto: el resultado no se vuelve a traducir a lenguaje humano, sino que la JVM ejecuta el bytecode. Además, JVM significa Java Virtual Machine.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[] datos = {7, 5, 3};
int[] alias = datos;
int i = 0;
while (i < 3) {
    System.out.println(alias[i] / 2);
    i++;
}
alias = new int[]{9, 8, 6};
System.out.println("B" + datos[0] + alias[0]);
double mitad = (double) (datos[1] / 2);
System.out.println(mitad);
```

Tu respuesta

```
3
2
1
B 7 9
2.5
```

Corrección. Las tres primeras líneas (3, 2, 1) son correctas. La cuarta tiene los valores correctos, 7 y 9, pero la salida exacta es B79, sin espacios, porque la concatenación no agrega separadores. La quinta debía ser 2.0: datos[1]/2 es la división entera 5/2 = 2, y el cast a double se aplica después; 2.5 solo tiene bien el formato decimal.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int modo = 1;
switch (modo) {
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
LISTO //
FIN
```

> **Corrección.** Clasificación y salida correctas: LISTO / FIN. Faltó explicar la regla: sin break, la ejecución sigue al case siguiente (fall-through).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
{ int nivel = 4; }
System.out.println(nivel);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
No está bien la síntaxis porque los corchetes lo encierran
```

> **Corrección.** Clasificación correcta: no compila. La causa se acerca, porque las llaves encierran la variable, pero no es un error de sintaxis: es de ámbito. nivel solo existe dentro del bloque que la declara, así que fuera de él no se conoce.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String etiqueta = null;
System.out.println(etiqueta.trim());
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
Sí compila porque esta bien planteado pero el .trim no se puede porque hay un null
```

Corrección. Correcto: compila, pero truena al ejecutarse porque se llama trim() sobre una referencia null (NullPointerException).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados. Escribe colocar() y una consulta de ocupadas. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué es final y qué static: capacidad o fabricados? ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
static int intentoValido(int[] intentos, int clave)
main static

// construtor: Tiene el nombre de la clase y es muy importante escribir `new` para que el
programa identifique que es un nuevo objeto, luego escribir la direccion, por ejemplo:
this.carro=33;
static [tipoDeDato] [NombreDeLaClase]{
this.[atributo1]= new[atributo1]

// forma de metodo (funcion)
[dato que da] [nombre del metodo]([tipo de parametro 1] [nombre del parametro 1] ...);
```

Justificaciones

```
los atributos de instancia están dentro de un método y son de una cifra, os atributos de
constante final ya no pueden modificarse y por lo mismo se escriben en mayuculas y con guiones
bajos.
Los constructores sirven para

Reto. ¿Qué es final y qué static: capacidad o fabricados? ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y
métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un
elemento sin justificar vale la mitad.
```

Corrección. No se modeló el cargador. No hay atributos capacidad, ocupadas ni fabricados, ni constructor o métodos concretos (solo plantillas genéricas, con un constructor marcado static que usa new por error), y no se contestó el reto. El modelo esperado tenía final int capacidad y int ocupadas de instancia, static int fabricados = 0, un constructor con this.capacidad = capacidad y fabricados++, colocar() con ocupadas++ y una consulta que devuelve ocupadas. Capacidad es final y fabricados es static; ninguno necesita ambos.

R4 · Código con repetición

Pregunta

El arreglo intentos contiene exactamente tres códigos enteros, en el orden en que se probaron. Devuelve el número del primer intento que coincide con clave (1, 2 o 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar al acertar. El arreglo no es null. No lo modifiques.

```
static int intentoValido(int[] intentos, int clave)
```

Ejemplos (argumentos → retorno):

{13, 42, 8}, 42 → 2 · {42, 8, 9}, 42 → 1 · {1, 2, 3}, 42 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int intentoValido(int[] intentos, int clave)

System.out.println(mitad);
```

Corrección. Solo está la cabecera correcta. Faltó el cuerpo: un do-while con un índice que compare intentos[i] == clave, devuelva i + 1 al acertar, avance mientras i < 3 y devuelva 0 al final. La línea System.out.println(mitad) no tiene relación con el problema e imprime, cosa que el enunciado prohibía.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Devuelve la acción del módulo con switch sobre Modo. Primero: si bateria < 10, devuelve "CARGAR", incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método. No imprimas ni leas del teclado.

```
enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
static String accion(Modo modo, int bateria)
```

Regla o valor recibido	Texto devuelto
APAGADO	"DETENER"
MEDIR	"LEER"
CALIBRAR	"AJUSTAR"
PRUEBA	"SIMULAR"

Tu respuesta

```
static String accion(Modo modo, int bateria)

int modo = NULL{
    if (bateria < 10){
        System.out.println("CARGAR");
            } else if {System.out.println("SIN MODO");
                }
enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
    this.apagado
    this.

switch (modo) {
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
}
```

Corrección. Bien: la cabecera, y revisar primero bateria < 10 para CARGAR, antes del switch. Hay tres problemas: se imprime en vez de devolver, el caso null no se revisa (el else if no tiene condición) y el switch es sobre un int con case 1/2, cuando debía ser sobre Modo con APAGADO→DETENER, MEDIR→LEER, CALIBRAR→AJUSTAR y PRUEBA→SIMULAR. La sintaxis no se reconoce (int modo = NULL{, this.apagado).

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.